

# **THENI KAMMAVAR SANGAM COLLEGE OF TECHNOLOGY**

## **PROJECT TITLE:**

**WhatNext Vision Motors: Shaping the Future of  
Mobility with Innovation and Excellence**

## **PROJECT DOMAIN:**

**Salesforce- Customer Relationship Management**

## **Department of Information Technology**

### **Team Leader:**

Priyanka Devi V – 922223205901

### **Team Members:**

Rajalaxmi P-922223205301

Pavithra Devi S-922223205033

Rishika R-922223205039

Priyanga Sobana C-922223205036

# **WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence**

## **User Story:**

WhatNext Vision Motors is redesigning its after-sales experience through an EV Service Hub built on Salesforce CRM. Instead of treating a service request as a phone call and a spreadsheet row, the new solution captures battery-health checks, charging-bay reservations, warranty eligibility, parts availability, technician assignment, and customer notifications in one traceable workflow. A customer can request help from a portal, the system checks the vehicle and preferred location, routes the work to the best-equipped service center, and keeps every status visible to the customer and service advisor. The implementation uses record-triggered flows for routing, Apex for warranty and stock rules, batch processing for dormant work orders, and scheduled automation for maintenance reminders. The result is a calmer service desk, more accurate promises, and a more transparent ownership experience.

## **1. INTRODUCTION**

### **1.1 Project Overview:**

The EV Service Hub centralizes vehicle service requests, charging support, warranty checks, and workshop capacity into Salesforce. It replaces disconnected emails and manual follow-ups with a guided service intake, capacity-aware assignment, live work-order statuses, and a shared customer timeline. Service teams can see which technicians, bays, parts, and appointment windows are available before making a promise.

### **1.2 Purpose:**

The purpose of this project is to make EV after-sales service predictable and data-driven. The system must reduce repeat calls, prevent appointments from being created when the required bay or part is unavailable, and give customers clear progress updates. It also gives managers a reliable view of first-time-fix rate, average response time, charger downtime, and warranty exposure.

## 2. IDEATION PHASE

### 2.1 Problem Statement (Customer Problem Statement Template)

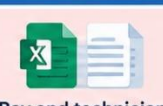
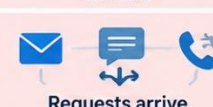
| PS   | I am<br>(Customer) | I'm trying to                 | But                             | Because                                         | Which makes<br>me<br>feel    |
|------|--------------------|-------------------------------|---------------------------------|-------------------------------------------------|------------------------------|
| PS-1 | Service Advisor    | Promise an appointment        | Capacity changes during the day | Bay and technician data live in separate sheets | Exposed to escalations       |
| PS-2 | EV Owner           | Get a charging issue resolved | Status updates are vague        | There is no shared case timeline                | Uncertain and frustrated     |
| PS-3 | Workshop Manager   | Plan parts and labour         | Demand is not visible early     | Requests arrive through different channels      | Reactive instead of prepared |



## 2. IDEATION PHASE

### 2.1 Problem Statement (Customer Problem Statement Template)

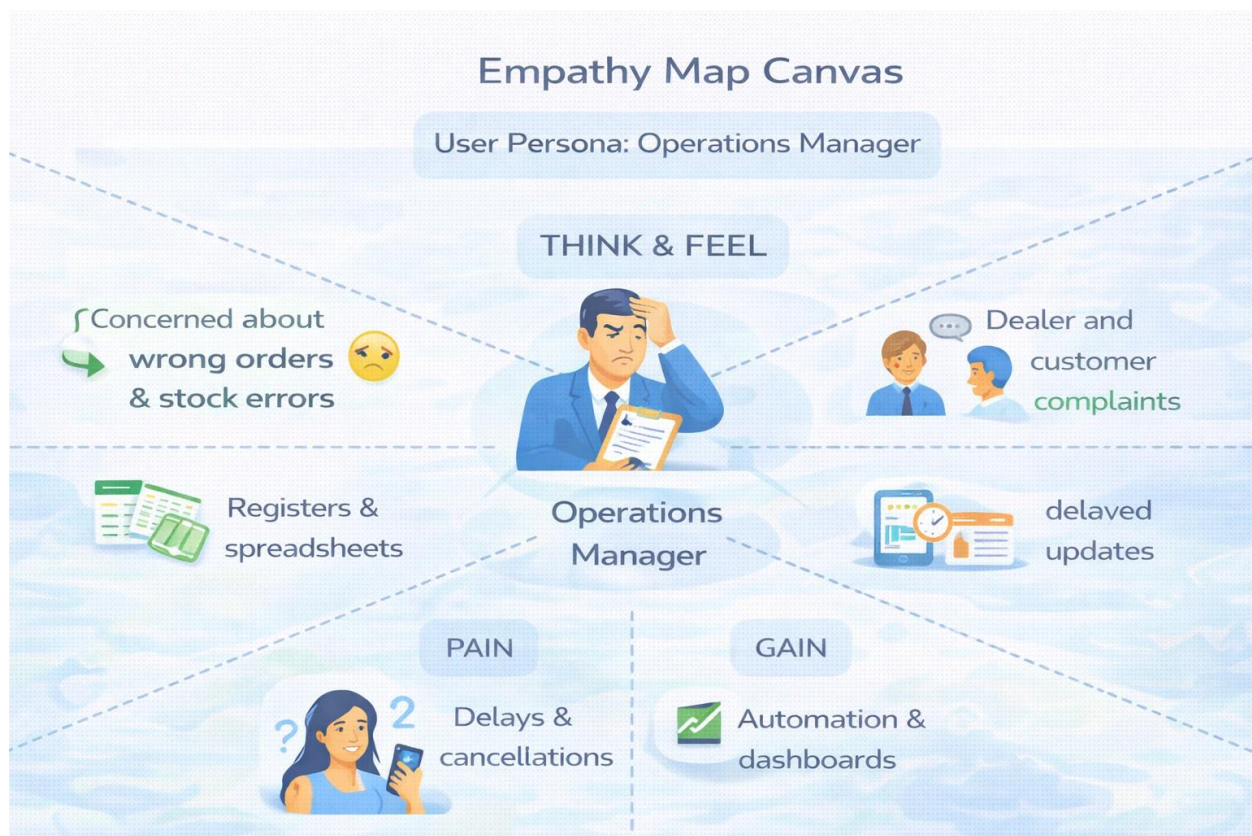


| PS   | I am<br>(Customer)                                                                                          | I'm trying to                                                                                                            | But                                                                                                                        | Because                                                                                                                                      | Which makes me feel                                                                                                       |
|------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| PS-1 |  <p>Service Advisor</p>  |  <p>Promise an appointment</p>        |  <p>Capacity changes during the day</p> |  <p>Bay and technician data live in separate sheets</p> |  <p>Exposed to escalations</p>       |
| PS-2 |  <p>EV Owner</p>         |  <p>Get a charging issue resolved</p> |  <p>Status updates are vague</p>        |  <p>There is no shared case timeline</p>                 |  <p>Uncertain and frustrated</p>     |
| PS-3 |  <p>Workshop Manager</p> |  <p>Plan parts and labour</p>         |  <p>Demand is not visible early</p>     |  <p>Requests arrive through different channels</p>       |  <p>Reactive instead of prepared</p> |

## 2.2 Empathy Map Canvas

### User Persona: Service Advisor

- **Think & Feel:** Wants to give a realistic appointment instead of revising it later.
- **Hear:** Customers ask when a vehicle will be ready.
- **See:** Chat messages, workshop boards, parts alerts, and warranty notes.
- **Say & Do:** Collects the VIN, checks capacity, calls the workshop, and follows up manually.
- **Pain:** Duplicate requests and unclear ownership of the next action.
- **Gain:** A guided intake, visible SLA timers, and automatic updates.



## 2.3 Brainstorming & Idea Prioritization

### Ideas Considered

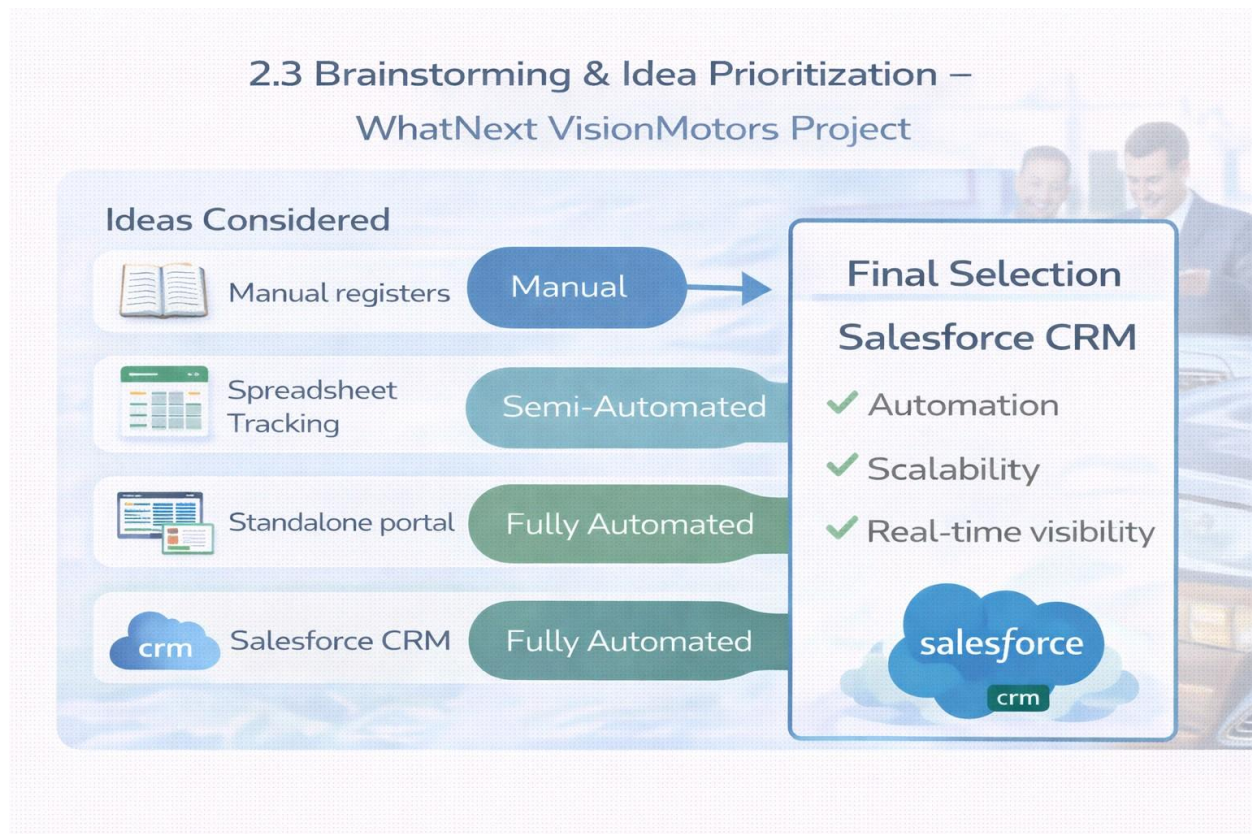
- Keep the existing call-and-spreadsheet process.
- Create a lightweight booking form only.
- Use a separate ticketing tool for service requests.
- Build an EV Service Hub in Salesforce with portal, automation, and dashboards.



## Final Selection

Salesforce CRM selected due to:

- A shared record for vehicle, customer, warranty, work order, and appointment.
- Declarative automation for routing and customer communication.
- Extensible Apex for warranty, parts, and capacity rules.
- Role-based visibility for advisors, technicians, managers, and customers.



## 3.REQUIREMENT ANALYSIS PHASE

### 3.1 Customer Journey Map

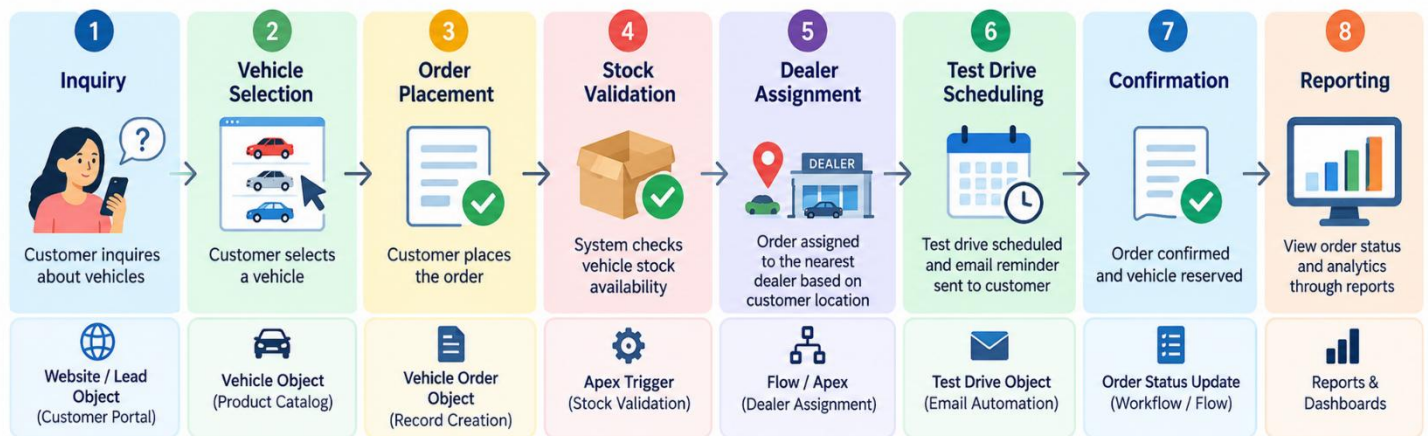
Stages:

Issue Reported → Vehicle Verification → Symptom Capture → Capacity Check → Service Center

Assignment → Appointment Confirmation → Repair Updates → Closure & Feedback

## 3.1 CUSTOMER JOURNEY MAP

### Vehicle Ordering Process Using Salesforce CRM



## 3.2 Solution Requirements

### Functional Requirements

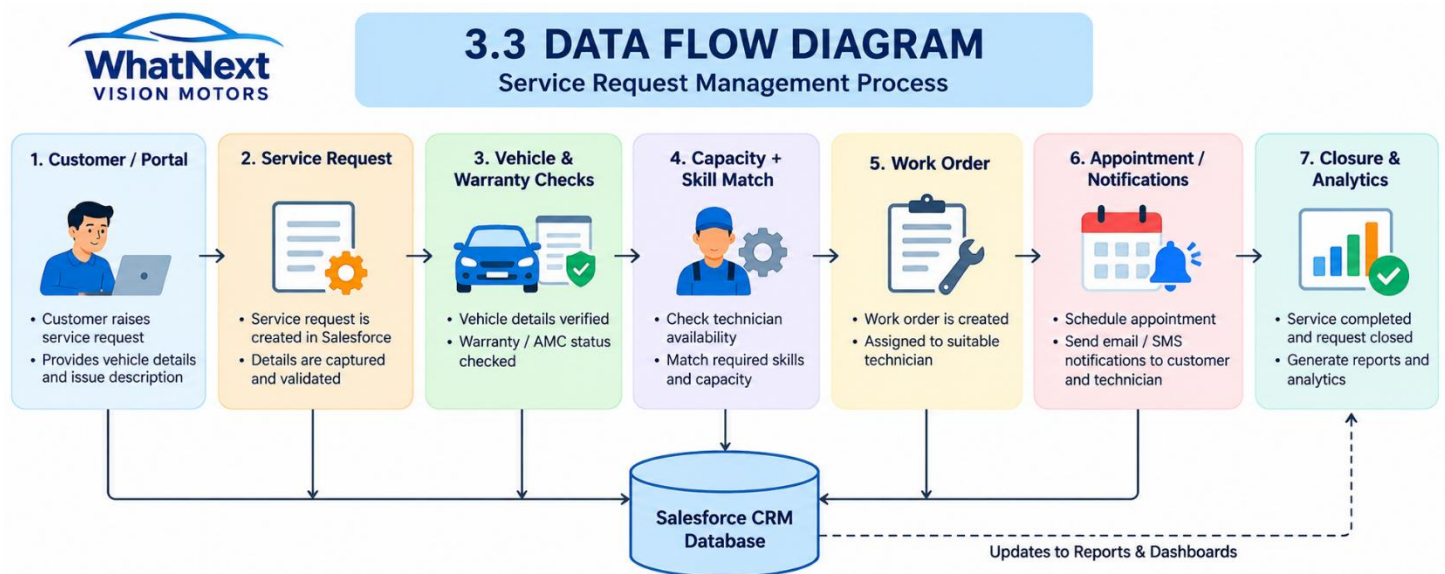
| FR No | Requirement                                                 |
|-------|-------------------------------------------------------------|
| FR-1  | Vehicle and battery-profile management                      |
| FR-2  | Service centre, bay, technician, and skill management       |
| FR-3  | Customer service request creation from portal and console   |
| FR-4  | Warranty, recall, and parts availability validation         |
| FR-5  | Capacity-aware service centre assignment                    |
| FR-6  | Appointment scheduling and reminder notifications           |
| FR-7  | Work-order analytics, SLA dashboards, and customer feedback |

## Non-Functional Requirements

| NFR          | Description                                                         |
|--------------|---------------------------------------------------------------------|
| Usability    | Guided Lightning intake with clear next actions                     |
| Security     | Sharing rules by service centre and role                            |
| Reliability  | Idempotent routing and duplicate-request protection                 |
| Performance  | A new request receives an assignment decision in under five seconds |
| Availability | Cloud access across service centres and mobile technicians          |
| Scalability  | Support for additional models, regions, and charging networks       |

### 3.3 Data Flow Diagram

Flow: Customer / Portal → Service Request → Vehicle & Warranty Checks → Capacity + Skill Match → Work Order → Appointment / Notifications → Closure Analytics





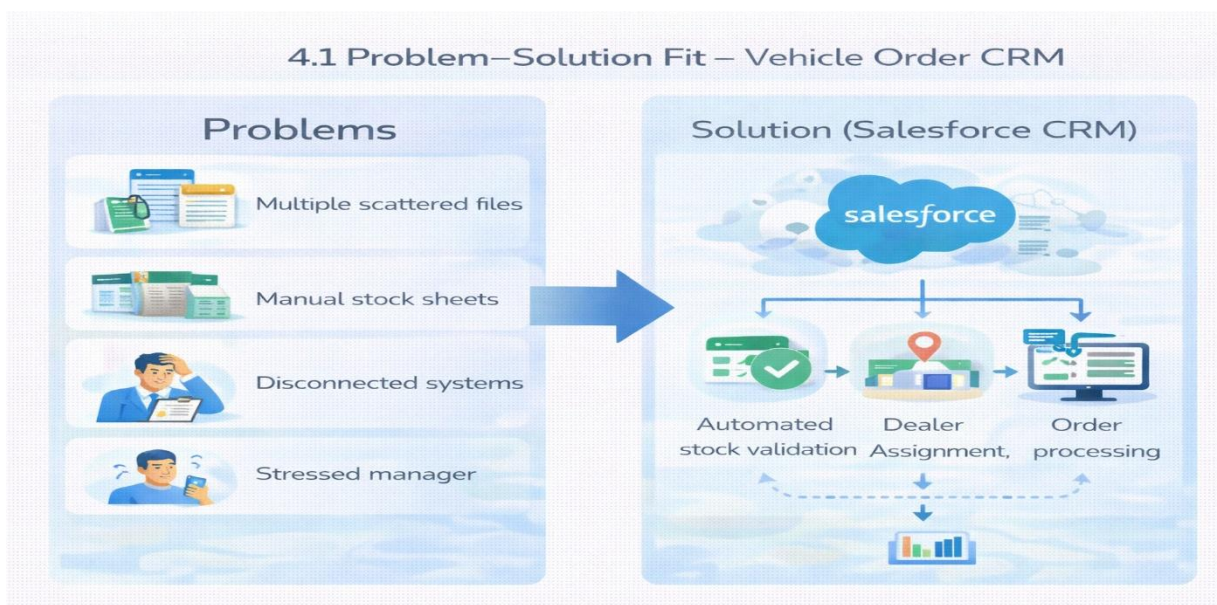
### 3.4 Technology Stack

| Layer      | Technology                                                  |
|------------|-------------------------------------------------------------|
| Experience | Salesforce Lightning App + Experience Cloud                 |
| Logic      | Record – Triggered Flows and Screen Flows                   |
| Database   | Custom Salesforce Objects and relationship fields           |
| Automation | Validation Rules, Apex Trigger Handler, Platform Events     |
| Processing | Batch Apex for stale requests, Scheduled Apex for reminders |
| Reporting  | Service dashboards, SLA reports, and feedback trends        |

## 4. PROJECT DESIGN PHASE

### 4.1 Problem–Solution Fit

The solution connects service intake, workshop capacity, parts readiness, and customer communication around one service request. The design deliberately separates the customer-facing request from internal work orders, so advisors can update the promise without losing the original customer language or diagnostic evidence.





## 4.2 Proposed Solution

### Functional Requirements

| Parameter      | Description                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------|
| Problem        | EV service demand is distributed across calls, messages, and spreadsheets                                 |
| Solution       | Salesforce EV Service Hub with guided intake and service console                                          |
| Innovation     | Capacity-aware routing using model, skill, bay, and parts signals                                         |
| Impact         | Fewer reschedules, faster first response, and clearer customer updates                                    |
| Business Model | Connected after-sales service across dealer and certified repair networks                                 |
| Scalability    | Add mobile technicians, new charging partners, and regional rules without redesigning the core data model |

### 4.3 Solution Architecture Components:

- Vehicle Object
- Dealer Object
- Customer Object
- Order Object
- Test Drive Object
- Service Request Object
- Flows
- Apex Triggers
- Batch Jobs
- Reports
- Dashboards

### 3. SOLUTION ARCHITECTURE

#### WhatNext Vision Motors – Salesforce CRM



## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

#### Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

| Sprint   | Functional Requirement (Epic) | User Story Number | User Story / Task                                                                                                                                                                     | Story Points | Priority | Team Members    |
|----------|-------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------|-----------------|
| Sprint-1 | Developer Setup               | USN-1             | As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.                          | 3            | High     | Rajalaxmi       |
| Sprint-2 | Data Modelling                | USN-2             | As an admin, I want to create Vehicle, Dealer, Customer, Order, and Service Request objects with proper relationships so that all business data is stored and managed systematically. | 5            | High     | Rishika         |
| Sprint-2 | Data Modelling                | USN-3             | As a staff member, I want dedicated tabs and a Lightning App for all modules so that I can easily navigate and manage vehicle orders and dealer operations.                           | 5            | High     | Priyanga Sobana |
| Sprint-3 | Automation                    | USN-4             | As a system, I want to validate stock availability, mandatory fields, and order inputs so that invalid or incomplete vehicle orders are prevented.                                    | 3            | High     | Pavithra Devi   |
| Sprint-3 | Automation                    | USN-5             | As a sales coordinator, I want automated flows to assign the nearest dealer and update order status so that processing becomes faster and requires minimal manual effort.             | 3            | High     | Priyanka Devi   |
| Sprint-4 | Apex                          | USN-6             | As a system, I want Apex triggers to automatically perform actions such as order confirmation and dealer assignment so that critical processes occur instantly and accurately.        | 5            | High     | Rajalaxmi       |
| Sprint-4 | Apex                          | USN-7             | As an inventory manager, I want batch jobs to update vehicle stock levels automatically so that inventory data remains accurate and up to date.                                       | 5            | High     | Rishika         |
| Sprint-5 | Reports                       | USN-8             | As a manager, I want reports showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.                                | 4            | High     | Priyanga Sobana |
| Sprint-6 | Dashboards                    | USN-9             | As a manager, I want dashboards showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.                             | 4            | Medium   | Pavithra Devi   |

| Sprint   | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed (as on Planned End Date) | Sprint Release Date (Actual) |
|----------|--------------------|----------|-------------------|---------------------------|-------------------------------------------------|------------------------------|
| Sprint-1 | 20                 | 6 Days   | 06 Sep 2026       | 12 Sep 2026               | 12 Sep 2026                                     | 13 Sep 2026                  |
| Sprint-2 | 20                 | 5 Days   | 13 Sep 2026       | 18 Sep 2026               | 18 Sep 2026                                     | 19 Sep 2026                  |
| Sprint-3 | 20                 | 6 Days   | 19 Sep 2026       | 24 Sep 2026               | 24 Sep 2026                                     | 25 Sep 2026                  |
| Sprint-4 | 20                 | 5 Days   | 25 Sep 2026       | 29 Sep 2026               | 29 Sep 2026                                     | 30 Sep 2026                  |

### Project Tracker, Velocity & Burndown Chart: (4 Marks)

## 6. Project Development Phase:

### What you'll learn

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

## Milestone 1: Salesforce Account

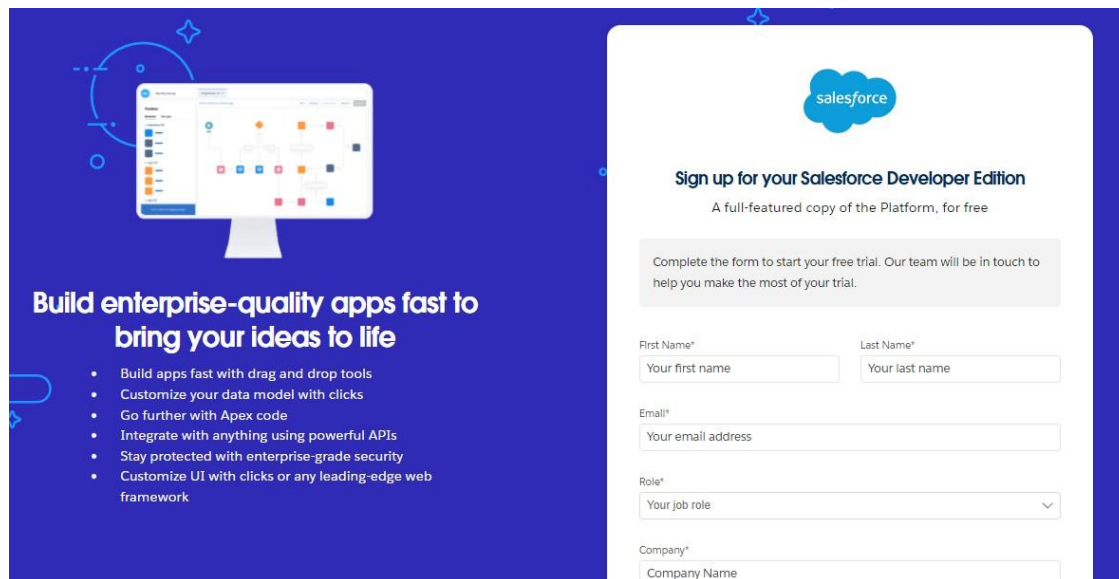
### Activity 1: Creating Developer Account

Creating a developer org in salesforce. 1. Go to

<https://developer.salesforce.com/signup>

2. On the sign up form, enter the following details :





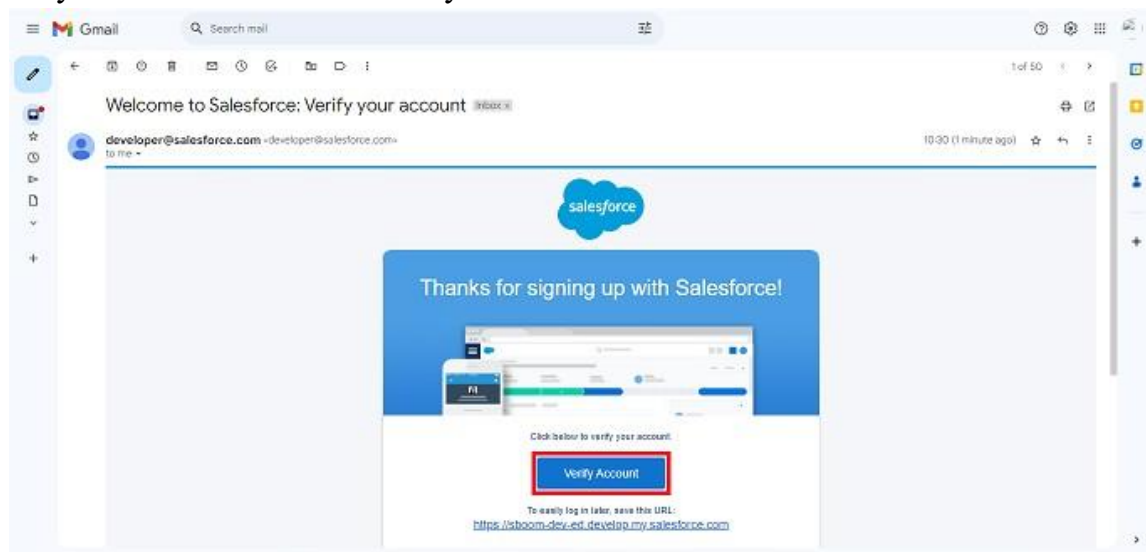
- 1) First name & Last name
- 2) Email
- 3) Role : Developer
- 4) Company : College Name
- 5) Country : India
- 6) Postal Code : pin code
- 7) Username : should be a combination of your name and company This need not

be an actual email id, you can give anything in the format :

[username@organization.com](mailto:username@organization.com) Click on sign  
me up after filling these.

## Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.

Change Your Password

Enter a new password for lead@sb.com.  
Make sure to include at least:

- 8 characters
- 1 letter
- 1 number

\* New Password

..... Good

\* Confirm New Password

..... Match

Security Question

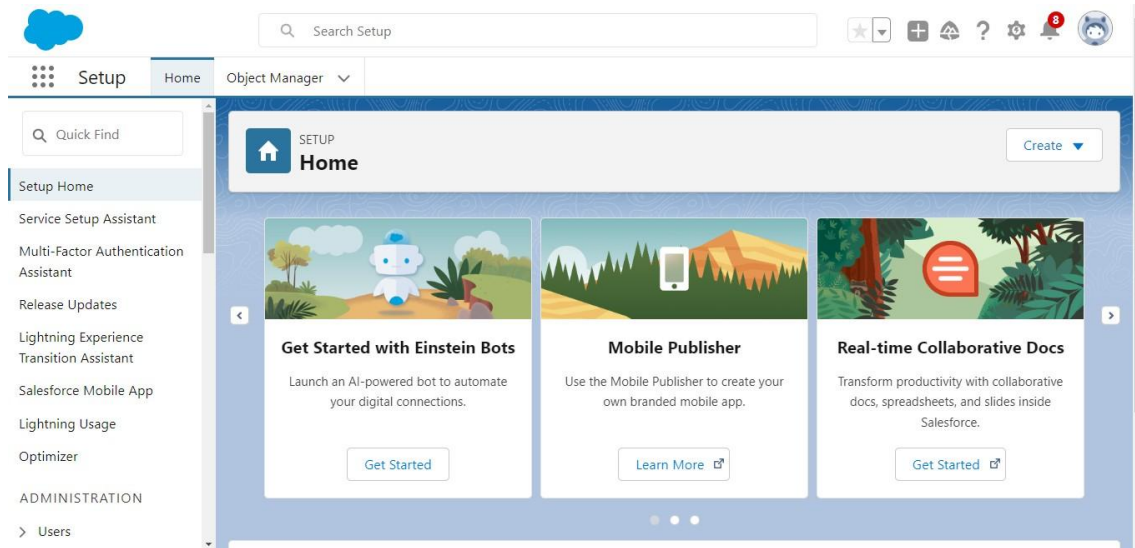
▼ In what city were you born?

\* Answer

asdfghjkl

Change Password

4. Then you will redirect to your salesforce setup page.



## Milestone 2: Data Management-Objects

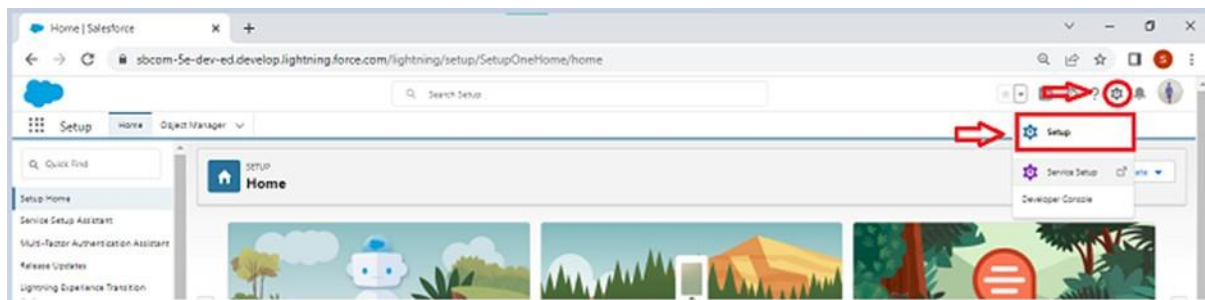
### Objects & Relationships

| Object Name         | Purpose                       | Relationships                   |
|---------------------|-------------------------------|---------------------------------|
| Vehicle__c          | Stores vehicle details        | Related to Dealer & Orders      |
| Vehicle_Dealer__c   | Stores authorized dealer info | Related to Orders               |
| Vehicle_Customer__c | Stores customer details       | Related to Orders & Test Drives |

|                              |                            |                               |
|------------------------------|----------------------------|-------------------------------|
| <b>Vehicle_Order__c</b>      | Tracks vehicle purchases   | Related to Customer & Vehicle |
| <b>Vehicle_Test_Drive__c</b> | Tracks test drive bookings | Related to Customer & Vehicle |

## To Navigate to Setup page:

Click on gear icon → click setup.



## Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



- 1) Enter the label name → Vehicle
- 2) Plural label name → Vehicles
- 3) Enter Record Name Label and Format
  - Record Name → Vehicle Name
  - Data Type → Text

**Custom Object Definition Edit** [Save] [Save & New] [Cancel]

**Custom Object Information**

The singular and plural labels are used in [tabs, page layouts, and reports](#).

Label  Example: Account

Plural Label  Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name  Example: Account

Description

Context-Sensitive Help Setting ☒ Open the standard Salesforce.com Help & Training window  
☐ Open a window using a Visualforce page

Content Name

**Enter Record Name Label and Format**

The Record Name appears in page layouts, [key lists, related lists, lookups, and search results](#). For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is required.

Record Name  Example: Account Name

Data Type  **Warning:** If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

2. Click on Allow reports,
3. Allow search → **Save**.

**Optional Features**

☒ Allow Reports

☐ Allow Activities

☐ Track Field History

☐ Allow in Chatter Groups

☐ Enable Licensing

**Object Classification**

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

☒ Allow Sharing

☒ Allow Bulk API Access

☒ Allow Streaming API Access

**Deployment Status** [What is this?](#)

☐ In Development

☒ Deployed

**Search Status**

When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

☒ Allow Search

**Object Creation Options (Available only when custom object is first created)**

☐ Add Notes and Attachments related list to default page layout

☐ Launch New Custom Tab Wizard after saving this custom object

[Save] [Save & New] [Cancel]

## Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
  - 1) Enter the label name→ Vehicle Dealer
  - 2) Plural label name→ Vehicle Dealers
  - 3) Enter Record Name Label and Format
    - Record Name → Dealer Name
    - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

### Activity 3: Creating remaining objects

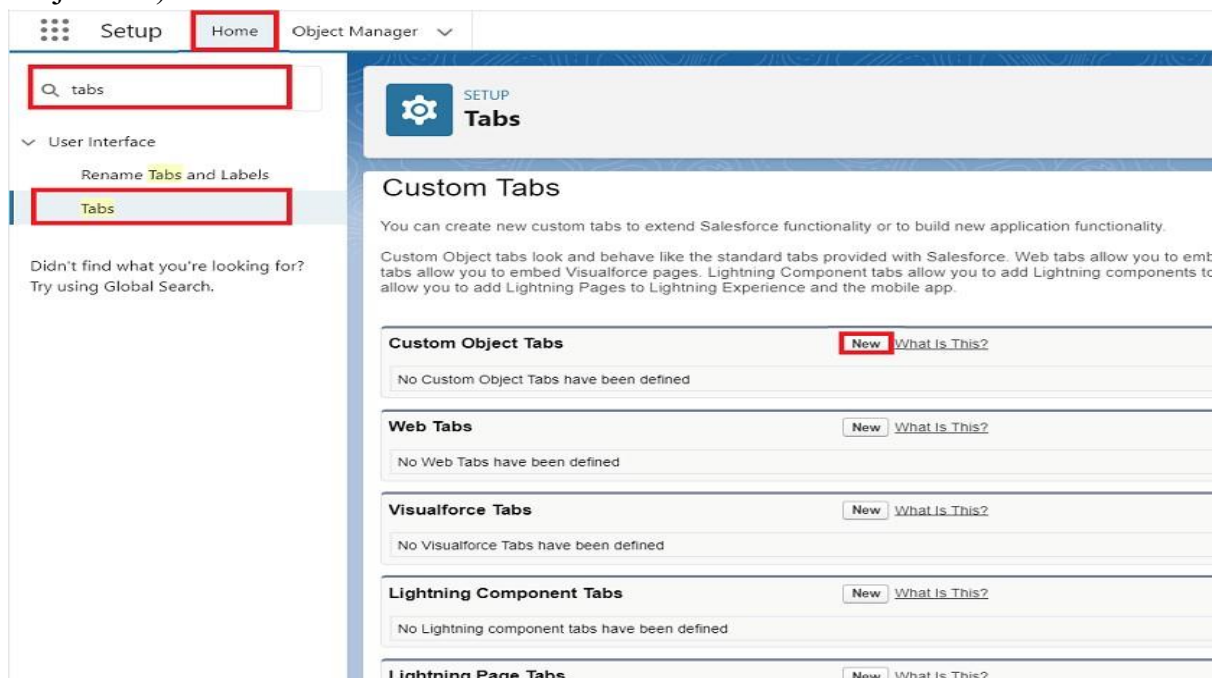
#### Creating remaining objects

Create remaining objects which have mentioned in the above table Data Management-Objects

## Milestone 3: Data Management-Tabs

### Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



Setup Home Object Manager

Search tab

Feature Settings

Analytics

Tableau

Tableau Embedding

Decision Tables

User Interface

Loaded Console Tab Limit

Rename Tabs and Labels

Tabs

Didn't find what you're looking for? Try using Global Search.

SETUP Tabs

New Custom Object Tab

Step 1 of 3

Choose the custom object for this new custom tab. Fill in other details.

New Custom Object Tab

Select an existing custom object or create a new custom object now

Object: --None--

Tab Style: --None--

(Optional) Choose a Home Page Custom Link to show as a splash page the first time your users click on this tab.

Splash Page Custom Link: --None--

Enter a short description.

Description

Next Cancel

## Activity 2:

Now create tabs for Other objects as well which we have created above Objects.

## Milestone 4: Data Management-App Manager

### Activity 1: Create a Lightning App To

create a lightning app page:

- Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.

App Manager | Salesforce

sbcom-5e-dev-ed.develop.lightning.force.com/lightning/setup/NavigationMenus/home

Search Setup

Setup Home Object Manager

app manager

Lightning Experience App Manager

Clone App (Beta)

Enable App Cloning

35 items • Sorted by App Name • Filtered by All app managers • TabSet Type

| App Name                         | Developer Name         | Description                                                                                       | Last Modified        | App Type            | Vis... |
|----------------------------------|------------------------|---------------------------------------------------------------------------------------------------|----------------------|---------------------|--------|
| 1. All Tabs                      | All Tabs               |                                                                                                   | 04/12/2022, 10:13 am | Classic             |        |
| 2. Analytics Studio              | Insights               | Build CRM Analytics dashboards and apps.                                                          | 04/12/2022, 10:13 am | Classic             |        |
| 3. App Launcher                  | App Launcher           | App Launcher tabs                                                                                 | 04/12/2022, 10:12 am | Classic             |        |
| 4. Best Solutions                | LightningBolt          | Discover and manage business solutions designed for your industry.                                | 04/12/2022, 10:13 am | Lightning           |        |
| 5. Chatter Desktop               | Chatter Desktop        | Chatter Desktop is an Adobe AIR-based desktop application that lets Chatter users stay connect... | 23/12/2022, 4:04 pm  | Connected (Managed) |        |
| 6. Chatter Mobile for BlackBerry | Chatter for BlackBerry | The Salesforce.com Chatter Mobile app lets you access Chatter data on the go. Use it to view...   | 23/12/2022, 4:05 pm  | Connected (Managed) |        |
| 7. College Management System     | nzeeem                 | demo app                                                                                          | 08/12/2022, 4:18 pm  | Lightning           |        |
| 8. Community                     | Community              | Salesforce CRM Communities                                                                        | 04/12/2022, 10:13 am | Classic             |        |
| 9. Content                       | Content                | Salesforce CRM Content                                                                            | 04/12/2022, 10:13 am | Classic             |        |
| 10. Data Manager                 | Data Manager           | Use Data Manager to view limits, monitor usage, and manage recipes.                               | 04/12/2022, 10:13 am | Lightning           |        |

- Fill the app name in app details and branding as follow

App Name : WhatNext Vision Motors

Developer Name : this will auto populated

Description : Give a meaningful description

Image : optional (if you want to give any image you can otherwise not mandatory)

Primary color hex value : keep this default

3. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.

New Lightning App

App Details & Branding

Give your Lightning app a name and description. Upload an image and choose the highlight color for its navigation bar.

**App Details**

\*App Name ⓘ  
Name your app...

\*Developer Name ⓘ  
Enter a developer name...

Description ⓘ  
Enter a description...

**App Branding**

Image ⓘ  
Upload

Primary Color Hex Value  
#007002

Org Theme Options  
☐ Use the app's image and color instead of the org's custom theme

App Launcher Preview

Next

4. To Add Navigation Items:

New Lightning App

Type to filter list...

Accounts

All Sites

Alternative Payment Methods

Analytics

App Launcher

Appointment Categories

Appointment Invitations

Approval Requests

Asset Action Sources

Asset Actions

HandsMen Products

Inventories

HandsMen Orders

Marketing Campaigns

Back

Next

Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

**Note:** select the custom object which we have created in the previous activity.

5. To Add User Profiles:

New Lightning App

User Profiles

Choose the user profiles that can access this app.

**Available Profiles**

System administrator

**Selected Profiles**

System Administrator

Save & Finish

Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

## **Milestone 5: Data Management-Fields**

### **Key Fields for Each Object**

#### **1. Vehicle\_\_c (Custom Object)**

- **Vehicle\_Name\_\_c** (Text)
- **Vehicle\_Model\_\_c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock\_Quantity\_\_c** (Number)
- **Price\_\_c** (Currency)
- **Dealer\_\_c** (Lookup to Dealer\_\_c)
- **Status\_\_c** (Picklist: Available, Out of Stock, Discontinued)

#### **2. Vehicle\_Dealer\_\_c (Custom Object)**

- **Dealer\_Name\_\_c** (Text)
- **Dealer\_Location\_\_c** (Text)
- **Dealer\_Code\_\_c** (Auto Number)
- **Phone\_\_c** (Phone)
- **Email\_\_c** (Email)

#### **3. Vehicle\_Order\_\_c (Custom Object)**

- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Order\_Date\_\_c** (Date)
- **Status\_\_c** (Picklist: Pending, Confirmed, Delivered, Canceled)

#### **4. Vehicle\_Customer\_\_c (Custom Object)**

- **Customer\_Name\_\_c** (Text)
- **Email\_\_c** (Email)
- **Phone\_\_c** (Phone)
- **Address\_\_c** (Text)
- **Preferred\_Vehicle\_Type\_\_c** (Picklist: Sedan, SUV, EV, etc.)

#### **5. Vehicle\_Test\_Drive\_\_c (Custom Object)**

- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Test\_Drive\_Date\_\_c** (Date)
- **Status\_\_c** (Picklist: Scheduled, Completed, Canceled)

## 6. Vehicle\_Service\_Request\_\_c (Custom Object)

- **Customer\_\_c** (Lookup to Customer\_\_c)
- **Vehicle\_\_c** (Lookup to Vehicle\_\_c)
- **Service\_Date\_\_c** (Date)
- **Issue\_Description\_\_c** (Text)
- **Status\_\_c** (Picklist: Requested, In Progress, Completed)

### Object Field Creation :

1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.

The screenshot shows the Salesforce Object Manager interface. At the top, there's a search bar with 'vehicle' entered. Below it, a table lists objects. The 'Vehicle' object is highlighted, showing its API name as 'Vehicle\_\_c' and type as 'Custom Object'. The last modified date is '13/02/2025' and it is deployed.

2. Now click on “Fields & Relationships” → New

The screenshot shows the 'Fields & Relationships' page for the 'Vehicle' object. The 'New' button is highlighted in red. Below the navigation tabs, there's a table with columns: FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIELD, and INDEXED. Two fields are listed: 'Created By' (Lookup(User)) and 'Last Modified By' (Lookup(User)).

3. Select Data type as “PickList”.

The screenshot shows the field type selection page. The 'Picklist' option is selected and highlighted with a red box. Other options include Checkbox, Currency, Date, Date/Time, Email, Geolocation, Number, Percent, Phone, Picklist (Multi-Select), and Text.

4. Click on Next

The screenshot shows the 'New Custom Field' page for the 'Vehicle' object. The 'Field Label' is 'Vehicle Model' and the 'Field Name' is 'Vehicle\_Model'. The 'Values' section shows a list of values: 'Sedan', 'SUV', 'EV', and 'etc'. The 'Next' button is highlighted in red.

## 5. Fill the above as following:

- Field Label: Vehicle Modal
- Fill PickList Values
- Click on Next → Next → Save and new.

## Object Field Creation-2 :

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.

The screenshot shows the 'Object Field Creation' interface. On the left, a sidebar lists various configuration options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Restriction Rules. The main area displays the 'Fields & Relationships' tab. Under the 'Details' section, the 'Number' data type is selected and highlighted with a red box. Other options include Checkbox, Currency, Date, Date/Time, Email, Geolocation, Percent, and Phone. Each option has a brief description of its functionality.

## 3. Click on Next

The screenshot shows the 'Step 2. Enter the details' form for creating a custom field. The 'Field Label' is 'Stock Quantity' and the 'Field Name' is 'Stock\_Quantity'. The 'Length' is set to 18 and 'Decimal Places' is 0. The 'Next' button is highlighted with a red box. The form also includes fields for 'Description' and 'Help Text'. The left sidebar lists various configuration options: Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Restriction Rules.

## 4. Fill the above as following:

- Field Label: Stock Quality
- Click on Next → Next → Save and new.

## Relationship Field Creation :

### Creating a Lookup RelationShip in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.

The screenshot shows the 'New Custom Field' form for creating a lookup relationship. The 'Lookup Relationship' data type is selected and highlighted with a red box. The 'Next' button is also highlighted with a red box. The form is titled 'Step 1. Choose the field type' and includes a section for 'Data Type' with options: None Selected, Auto Number, Formula, Roll-Up Summary, Lookup Relationship (selected), and Master-Detail Relationship. The left sidebar lists various configuration options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, and Scoping Rules.

2. Click Next
3. Select Dealer Object Form Related Filed.

**Step 3. Enter the label and name for the lookup field**

Field Label: Dealer

Field Name: Dealer

**Step 4. Establish field-level security for reference field**

| Field Label | Data Type | Field Name | Description |
|-------------|-----------|------------|-------------|
| Dealer      | Lookup    | Dealer     |             |

Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.

| Field Level Security for Profile | Visible                             | Read-Only                |
|----------------------------------|-------------------------------------|--------------------------|
| Analytics Cloud Integration User | <input checked="" type="checkbox"/> | <input type="checkbox"/> |
| Analytics Cloud Security User    | <input checked="" type="checkbox"/> | <input type="checkbox"/> |
| Contract Manager                 | <input checked="" type="checkbox"/> | <input type="checkbox"/> |
| Cross Org Data Proxy User        | <input checked="" type="checkbox"/> | <input type="checkbox"/> |
| Custom Marketing Profile         | <input type="checkbox"/>            | <input type="checkbox"/> |

4. Click Next and Save.

## More Fields Creation

Create remaining fields as mentioned in the Data Management-Fields

## Milestone 6: Automation

### Flow Creation :

**Creating a record triggered flow to assign nearest dealer to the customer's location Step 1 :**

In Quick Find, type Flows and click on Flows. Click New Flow.

**Setup** Home Object Manager

Search: flow

Apps

- Lightning Bolt
- Flow Category
- Process Automation
  - Automation Home (Beta)
  - Flows**
  - Migrate to Flow
  - Paused And Failed Flow
  - Interviews
  - Process Builder
- WorkFlow Actions

**SETUP Flows**

Flow Trigger Explorer New Flow

**Try the Automation Lightning App!**

These new features are available only in the Automation Lightning app:

- Search for automations
- Sort your list views with more options
- Organize your automations with categories and subcategories

If you don't see the app in the App Launcher, check that Enable the Automation Lightning App is selected in Process Automation Settings.

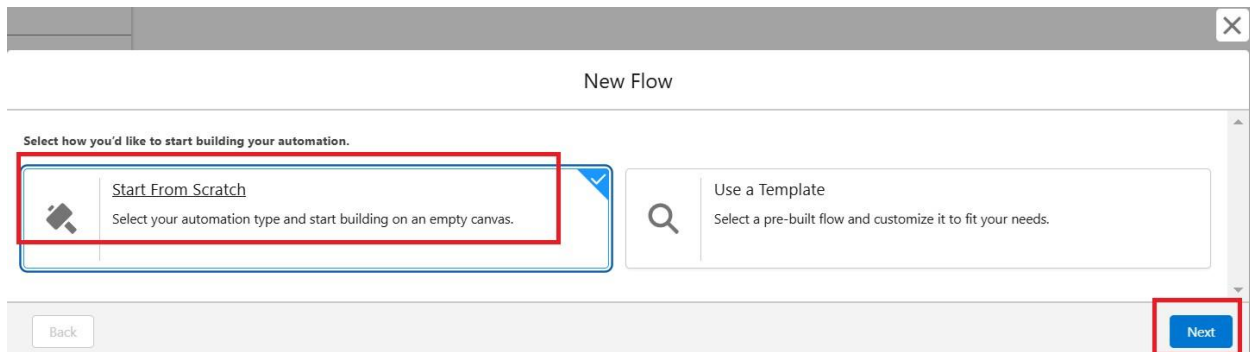
**Flow Definitions**

All Flows

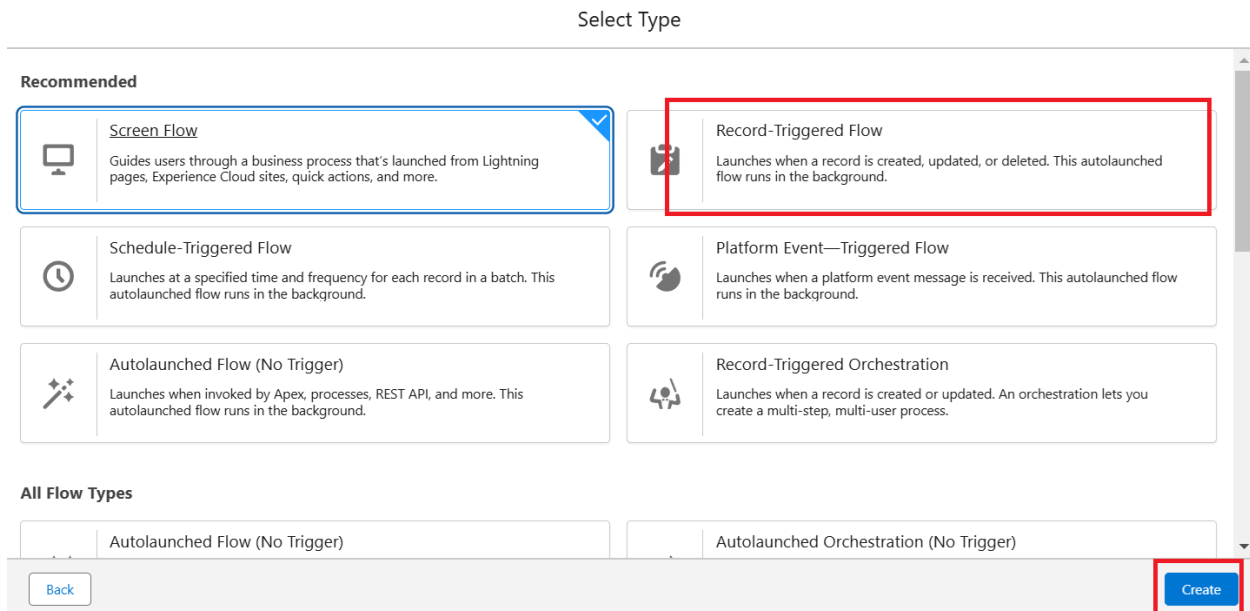
44 items • Sorted by Flow Label • Filtered by All flow definitions • Updated a few seconds ago



Step 2 : Select Start From Scratch and click Next.



Step 3 : Select Record-Triggered Flow and click Create



Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

**Set Entry Condition :**

All Conditions Are Met (AND)

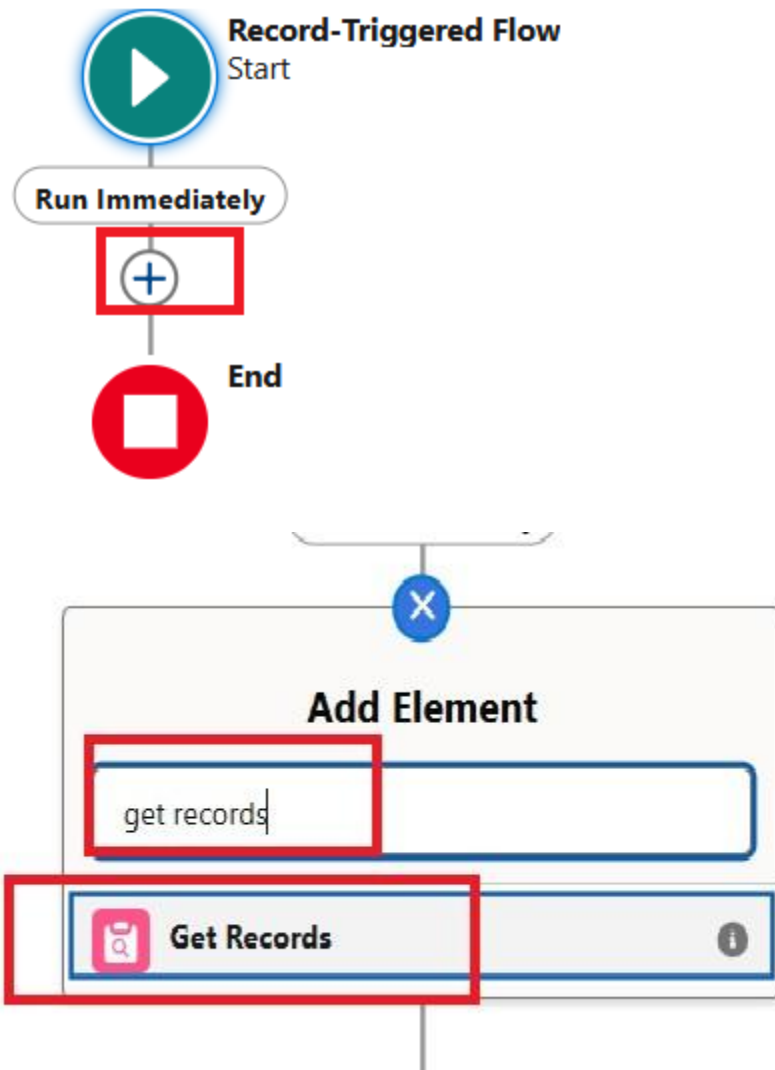
Filed : Status\_\_c

Operator : Equals

Value : Pending

```
app = Flask(__name__)
```

Step 5 : Click + → Select **Get Records**



#### Step 6 : **Label** : Get Customer Information

- Object : Vehicle Customer • Condition :
- Field : Id
- Operator : Equals
- Value : {!\$Record.Vehicle\_Customer\_\_c}

**Get Records**

\*Label:  \*API Name:

Description:

Get Records of This Object

\*Object:

Filter Vehicle Customer Records

Condition Requirements:

| Field                           | Operator                            | Value                                                        |
|---------------------------------|-------------------------------------|--------------------------------------------------------------|
| <input type="text" value="Id"/> | <input type="text" value="Equals"/> | <input type="text" value="{!\$Record.Vehicle_Customer__c}"/> |

[+ Add Condition](#)

Sort Vehicle Customer Records

## Step 7 : Click + → Select **Get Records**

- **Label** : Get Nearest Dealer
- **Object** : Vehicle Dealer
- **Condition** : All conditions Are Met (AND)
- **Field** : Dealer\_Location\_\_c
- **Operator** : Equals
- **Value** : {!Get\_Customer\_Information.Address\_\_c}



Get Records

\* Label

Get Nearest Dealer

\* API Name ⓘ

Get\_Nearest\_Dealer

Description

Get Records of This Object

\* Object

Vehicle Dealer

Filter Vehicle Dealer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field

Dealer\_Location\_\_c

Operator

Equals ▼

Value

Aa Vehicle Customer from Get\_Cu... X

+ Add Condition

## Step 8 : Click + → Select **Update Records**

- **Label** : Assign Dealer to Order
- **\*How to Find Records to Update and Set Their Values** : Use the IDs and all field values from a record or record collection
- **Select Record(s) to Update** : {!Get\_Nearest\_Dealer}

 Update Records >

\* Label

Assign Dealer to Order

\* API Name ⓘ

Assign\_Dealer\_to\_Order

Description

\* How to Find Records to Update and Set Their Values

☐ Use the vehicle order record that triggered the flow

☐ Update records related to the vehicle order record that triggered the flow

☒ Use the IDs and all field values from a record or record collection

☐ Specify conditions to identify records, and set fields individually

Select Record(s) to Update

Record or Record Collection

 Vehicle Dealer from Get Nearest Dealer ×

Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails. ⓘ

**Step 9 :** Click Save and Give label Name and Activate Flow.

- Label Name : Auto Assign Dealer

Save as

Save As

A New Version

\* Flow Label

Auto Assign Dealer

\* Flow API Name ⓘ

Auto\_Assign\_Dealer

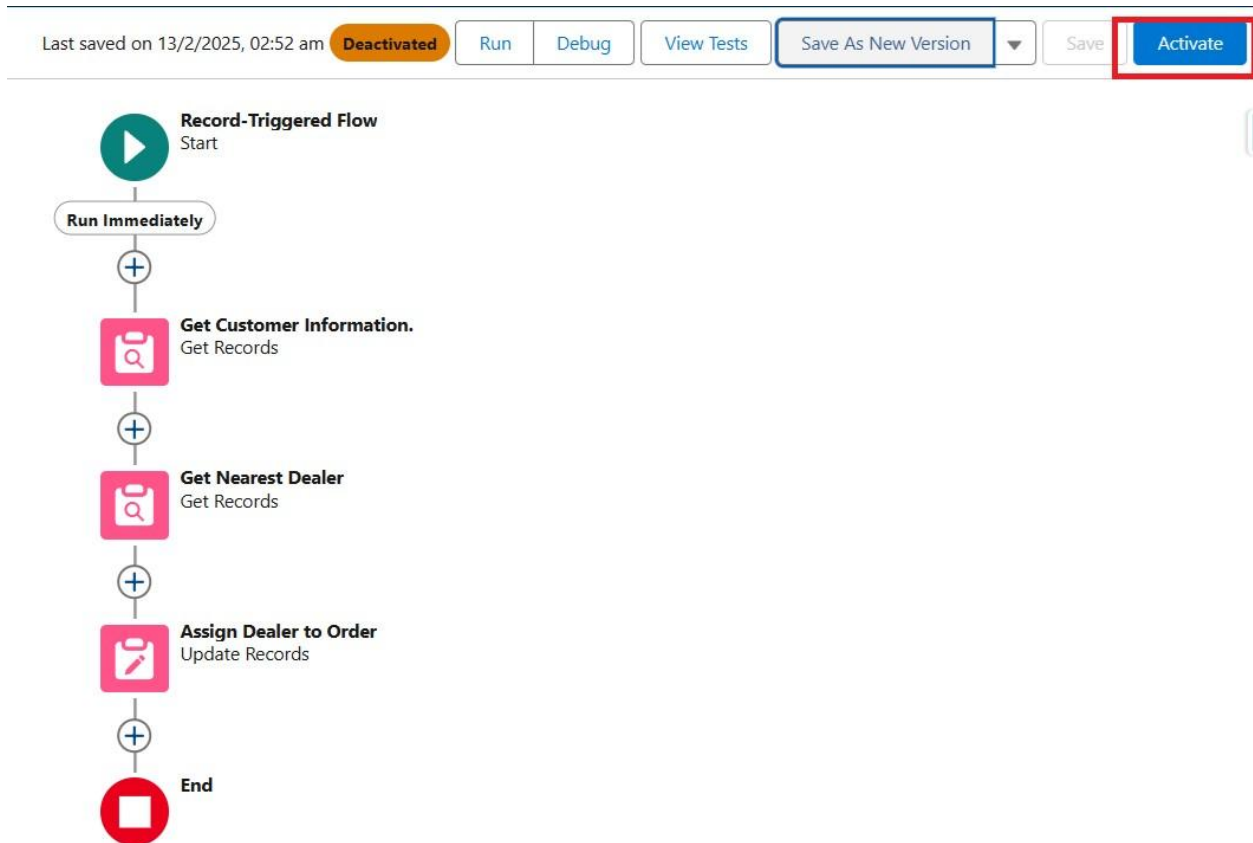
Description

[Show Advanced](#)

Cancel

Save

## Step 10 : Activate Flow



## Record-Trigger Flow Creation :

Creating a record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow and click Create

Select Type

| Recommended                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Screen Flow</b><br>Guides users through a business process that's launched from Lightning pages, Experience Cloud sites, quick actions, and more.    |  <b>Record-Triggered Flow</b><br>Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.                    |
|  <b>Schedule-Triggered Flow</b><br>Launches at a specified time and frequency for each record in a batch. This autolaunched flow runs in the background. |  <b>Platform Event—Triggered Flow</b><br>Launches when a platform event message is received. This autolaunched flow runs in the background.                |
|  <b>Autolaunched Flow (No Trigger)</b><br>Launches when invoked by Apex, processes, REST API, and more. This autolaunched flow runs in the background.   |  <b>Record-Triggered Orchestration</b><br>Launches when a record is created or updated. An orchestration lets you create a multi-step, multi-user process. |

All Flow Types

|                                                                                                                    |                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
|  Autolaunched Flow (No Trigger) |  Autolaunched Orchestration (No Trigger) |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|

[Back](#) [Create](#)

## Step 2 : Select Vehicle Test Drive Object

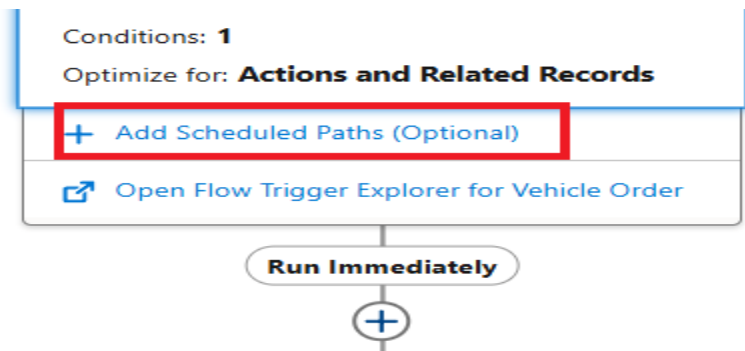
Trigger the Flow When : A record is created or updated

### Set Entry Condition :

- All Conditions Are Met (AND)
- Filed : Status\_\_c
- Operator : Equals
- Value : Scheduled

### Step 3 : Click + Add Scheduled Paths (below the trigger).

- Label: Reminder Before Test Drive.
- Time Source: Test\_Drive\_Date\_\_c
- Offset Number: 1.
- Offset Options: Days Before.
- Click Done.



**Configure Scheduled Paths**

**SCHEDULED PATHS** +

Run Immediately

Reminder Before Test Drive.

Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.

**SCHEDULED PATH DETAILS** [Delete Path](#)

\* Path Label:

\* API Name:

\* Time Source:

\* Offset Number:

\* Offset Options:

> Show advanced options



#### Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle\_Customer\_\_c.
4. Filter Conditions: Id = {!\$Record.Customer\_\_c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

**Get Records**

\* Label: Get Customer Information

\* API Name: Get\_Customer\_Information

Description

Get Records of This Object

Object: Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements: All Conditions Are Met (AND)

| Field | Operator | Value                       |
|-------|----------|-----------------------------|
| Id    | Equals   | \$Record > Vehicle Customer |

+ Add Condition

Sort Vehicle Customer Records

#### Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get\_Customer\_Information.Email\_\_c}
6. Rich-Text-Formatted Body : True



Send Email



\* Label

Send Test Drive Reminder.

\* API Name 

Send\_Test\_Drive\_Reminder

Description



**Send Email** 

emailSimple-emailSimple

7. Body : Create Variable

8 . Api Name : EmailSent



Send Email

 Body 



EmailSent



Included

 CC Recipient Address List



Not Included

 Email Template ID



Not Included

 Log Email on Send



Not Included

 Recipient Address Collection



Not Included

## Step 6 : Click save

- **Label Name** : Test Drive Reminder

Save as

---

Save As

A New Version

Flow Label

Test Drive Reminder

\* Flow API Name ⓘ

Test\_Drive\_Reminder

Description

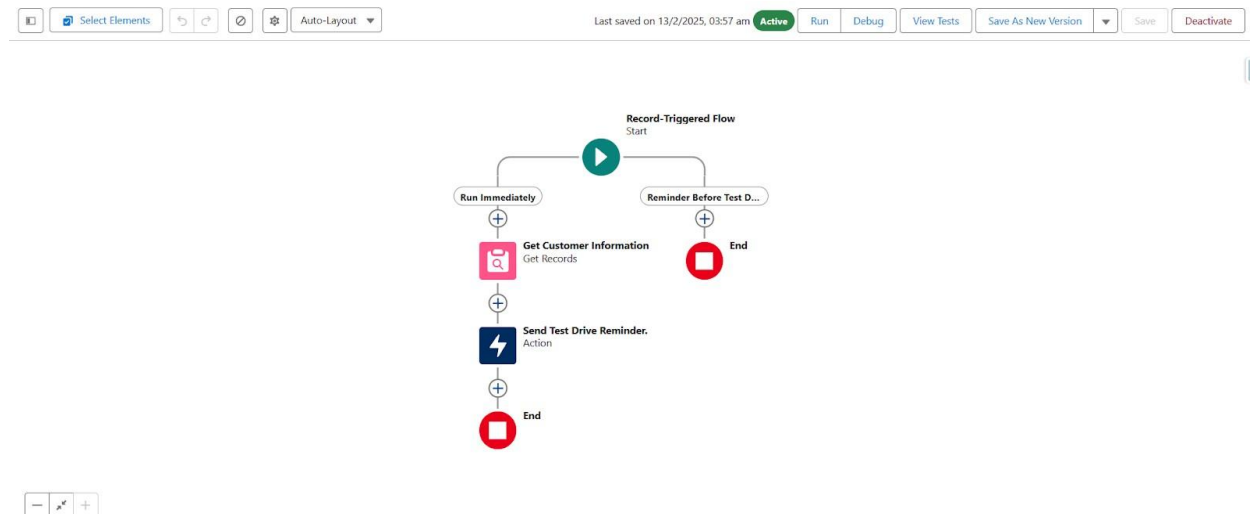
Show Advanced

Cancel Save

## Step 7 : Activate Flow

Run Debug View Tests Save As New Version Save Activate

## Flow Page After Creation :

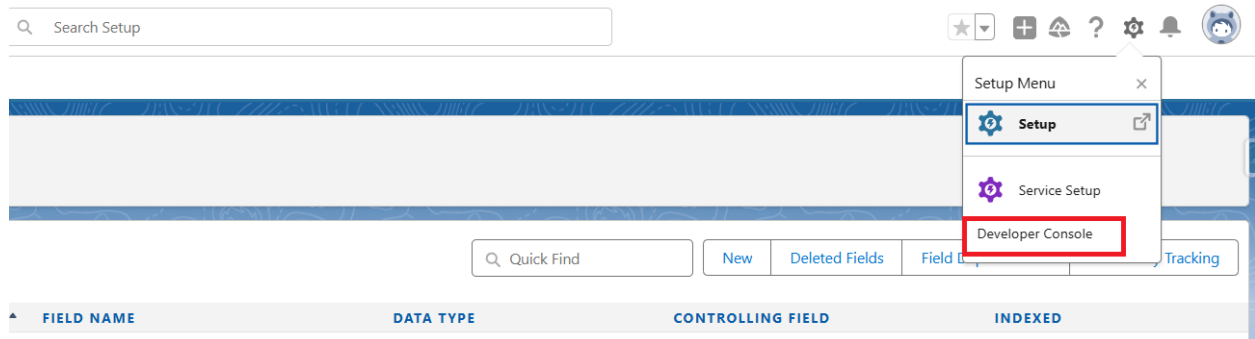




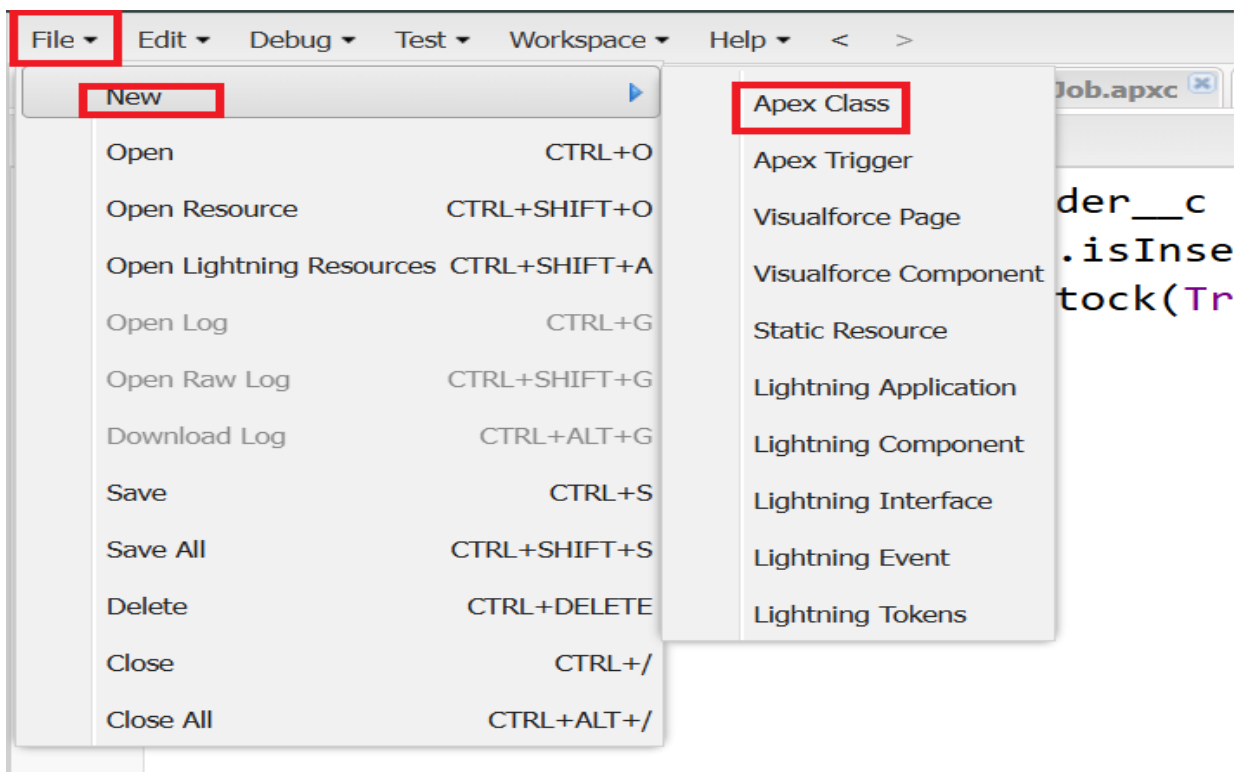
## Milestone 7: Apex and Batch Class

### Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

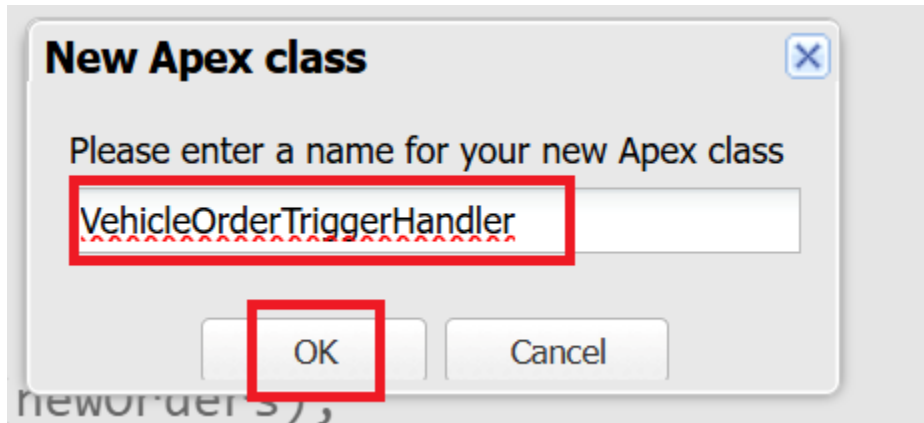


Step 2 : click File then Select New then Select Apex.



### Step 3 : Give Name For Apex Class

- Class Name : VehicleOrderTriggerHandler



### Step 4 : Write Apex Code

```
1 public class VehicleOrderTriggerHandler {
2
3     public static void handlertrigger(List<Vehicle_Order__c> neworders, Map<Id, Vehicle_Order__c> oldorders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4         if (isBefore) {
5             if (isinsert || isupdate) {
6                 preventOrderIfoutofstock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isinsert || isupdate) {
12                updateStockonorderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfoutofstock(List<Vehicle_Order__c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_Order__c order : orders) {
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29                vehicleStockMap.put(vehicle.Id, vehicle);
30            }
31
32            for (Vehicle_Order__c order : orders) {
33                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
34                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
35                    if (vehicle.Stock_Quantity__c <= 0) {
36                        order.addError('This vehicle is out of stock. Order cannot be placed.');
```

## Source Code :

```
public class VehicleOrderTriggerHandler {
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean
isUpdate) {

        if (isBefore) {

            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate)
            {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock    private static void
preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {        Set<Id> vehicleIds
= new Set<Id>();        for (Vehicle_Order__c order : orders) {            if
(order.Vehicle__c != null) {                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
```



```

        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {            vehicleStockMap.put(vehicle.Id, vehicle);

    }

    for (Vehicle_Order__c order : orders) {            if
(vehicleStockMap.containsKey(order.Vehicle__c)) {

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);            if
(vehicle.Stock_Quantity__c <= 0) {            order.addError('This vehicle is out of
stock. Order cannot be placed.');
```

```

    }

    }

    }

    }

    }

    // Method to update vehicle stock when an order is placed    private static void
updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {        Set<Id> vehicleIds
= new Set<Id>();        for (Vehicle_Order__c order : orders) {            if (order.Vehicle__c
!= null && order.Status__c == 'Confirmed') {            vehicleIds.add(order.Vehicle__c);

        }

    }

    if (!vehicleIds.isEmpty()) {

        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();

        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {            vehicleStockMap.put(vehicle.Id, vehicle);

    }

```

```

List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

for (Vehicle_Order__c order : orders) {
    if
(vehicleStockMap.containsKey(order.Vehicle__c)) {

        Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);

        if (vehicle.Stock_Quantity__c > 0) {
            vehicle.Stock_Quantity__c -
= 1;
            vehiclesToUpdate.add(vehicle);
        }
    }
}

if (!vehiclesToUpdate.isEmpty()) {
update vehiclesToUpdate;
}
}
}
}
}
}
}

```


Vehicle Order  
**VC-0003**

Vehicle Order Name

VC-0003

Owner

 Abdul Trainer

Vehicle Customer

 test KHIT 123434

×

Vehicle

 1erwe

×

Order Date

21/02/2025



Status

Pending

▼

Created By

 Abdul Trainer, 13/02/2025, 5:34 am

By

Trainer, 13/02/2025, 5:41 am

✕ We hit a snag.

Review the errors on this page.

- This vehicle is out of stock. Order cannot be placed.

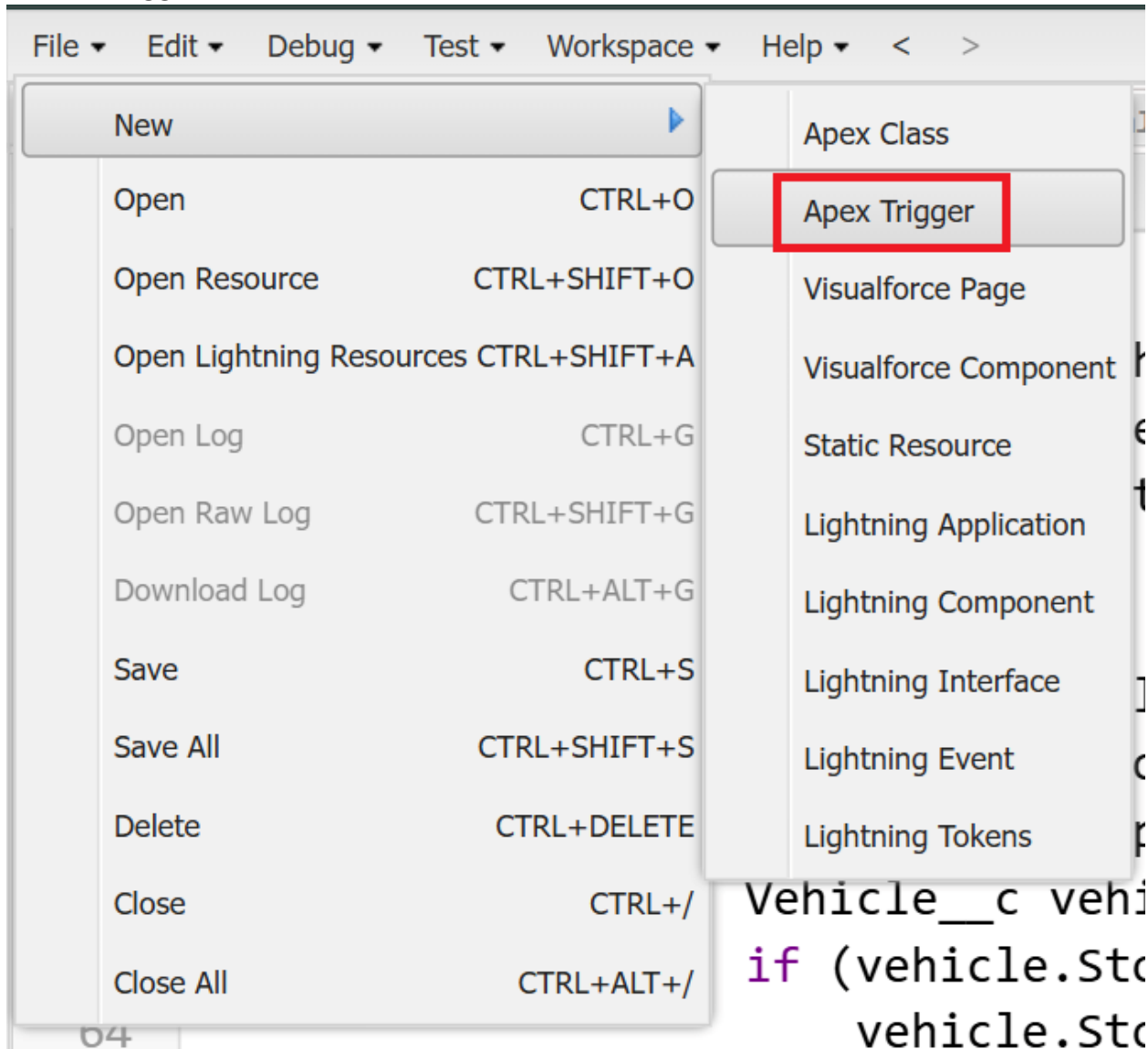
⛔

Cancel

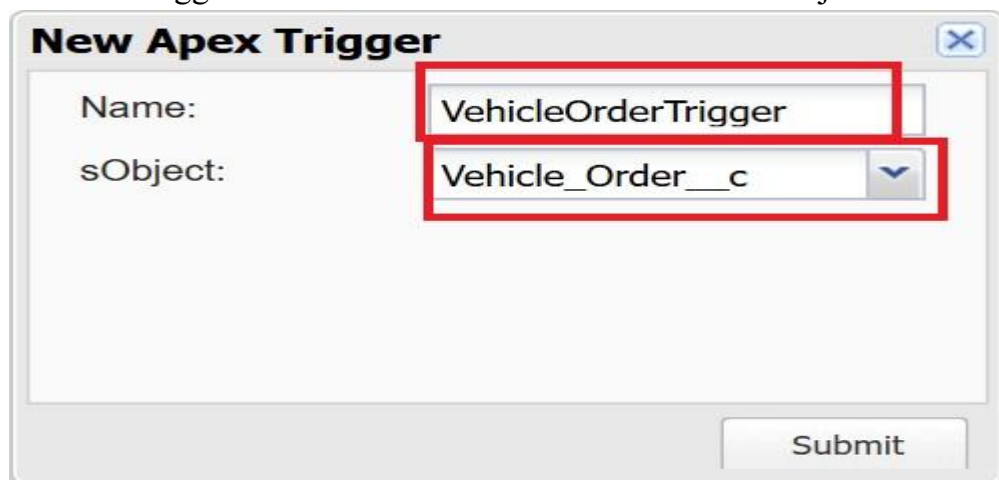
Save

\* = Required Information

## Step 5 : Write Trigger Handler



## Step 6 : Write Trigger Class Name and Select Vehicle Order Object



## Step 7 : Call Apex Class in Trigger Class

Source Code:

trigger VehicleOrderTrigger on Vehicle\_Order\_\_c (before insert,before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger([Trigger.new](#), Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate); }

```
Code Coverage: None | API Version: 62
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
2     VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
3 }
4
```

## Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.



```
Code Coverage: None | API Version: 62
1 global class VehicleOrderBatch implements Database.Batchable<Object> {
2
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4         return Database.getQueryLocator([
5             SELECT Id, Status__c, Vehicle__c
6             FROM Vehicle_Order__c
7             WHERE Status__c = 'Pending'
8         ]);
9     }
10
11     global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12         Set<Id> vehicleIds = new Set<Id>();
13
14         for (Vehicle_Order__c order : orderList) {
15             if (order.Vehicle__c != null) {
16                 vehicleIds.add(order.Vehicle__c);
17             }
18         }
19
20         if (!vehicleIds.isEmpty()) {
21             Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22             for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23                 vehicleStockMap.put(vehicle.Id, vehicle);
24             }
25
26             List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27             List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29             for (Vehicle_Order__c order : orderList) {
30                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
30  if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31      Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32      if (vehicle.Stock_Quantity__c > 0) {
33          order.Status__c = 'Confirmed';
34          vehicle.Stock_Quantity__c -= 1;
35          ordersToUpdate.add(order);
36          vehiclesToUpdate.add(vehicle);
37      }
38  }
39  }
40
41  if (!ordersToUpdate.isEmpty()) {
42      update ordersToUpdate;
43  }
44
45  if (!vehiclesToUpdate.isEmpty()) {
46      update vehiclesToUpdate;
47  }
48  }
49  }
50
51  global void finish(Database.BatchableContext bc) {
52      System.debug('Vehicle order batch job completed.');
```

53 }
54 }



The screenshot shows an IDE window with a code editor on top and a toolbar below. The code editor contains the code snippet above. The toolbar has several tabs: 'Logs', 'Tests', 'Checkpoints', 'Query Editor', 'View State', 'Progress', and 'Problems'. The 'Problems' tab is highlighted with a red rectangle. Below the toolbar, there is a table with columns 'User', 'Application', and 'Operation'.

## Source Code:

```
global class VehicleOrderBatch implements Database.Batchable<sObject>
{
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c
            'Pending'
        ]);
    }
    global void execute(Database.BatchableContext bc, List<Vehicle_Order_c>
    orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order_c order
        orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
    }
}
```

```

(!vehicleIds.isEmpty()) {
Map<Id, Vehicle___c> vehicleStockMap = new Map<Id, Vehicle___c>();

for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c WHERE Id IN
vehicleIds]) { vehicleStockMap.put(vehicle. Id, vehicle);

}

List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>();

List<Vehicle___c> vehiclesToUpdate = new List<Vehicle___c>(); for
(Vehicle_Order_c order orderList) { if (vehicleStockMap.containsKey(order.
Vehicle___c)) { Vehicle_c vehicle = vehicleStockMap.get(order. Vehicle___c); if
(vehicle.Stock_Quantity___c > 0) { order.Status___c = 'Confirmed';
vehicle.Stock_Quantity___c -= 1; ordersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);

}

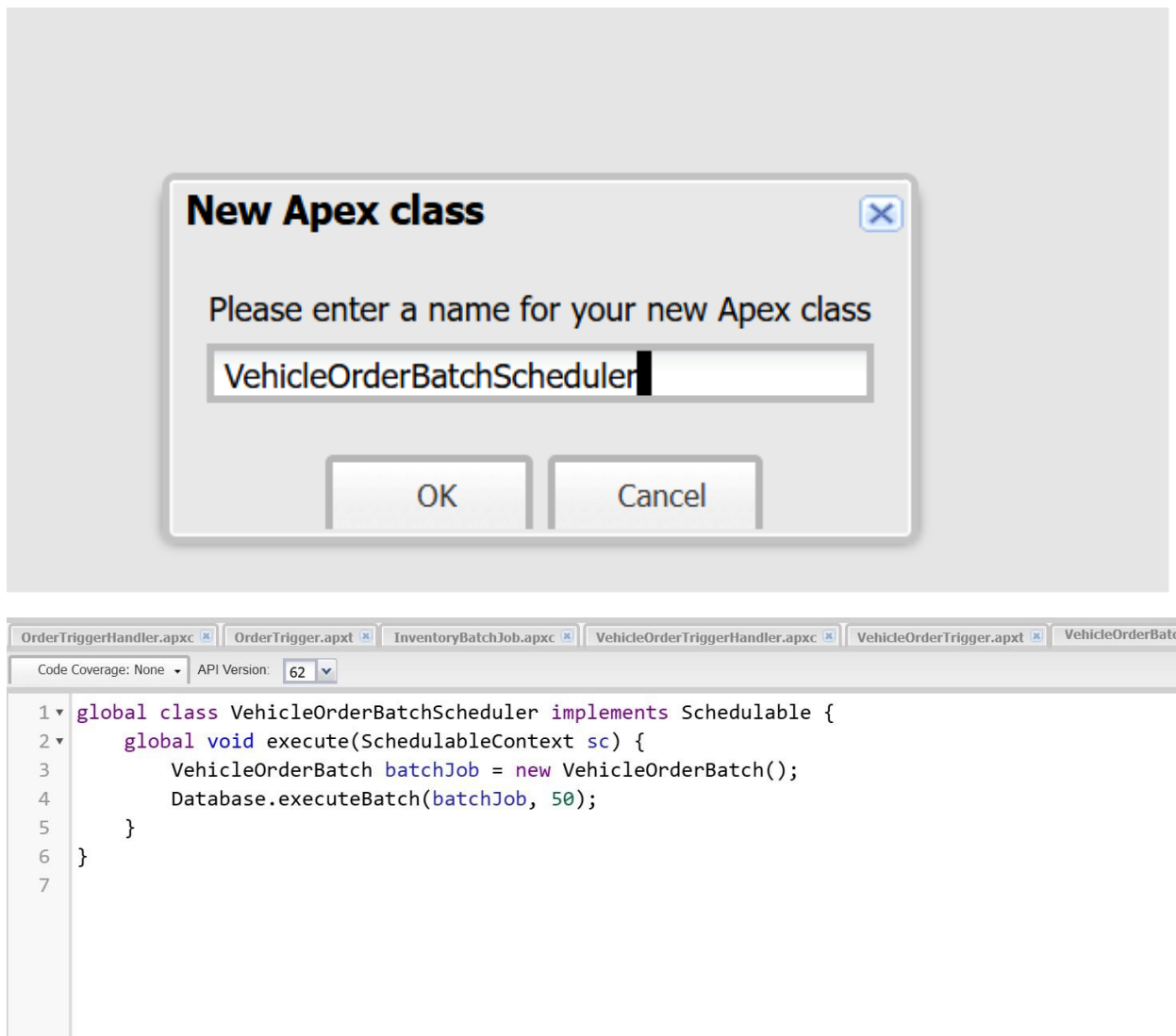
} } if (!ordersToUpdate.isEmpty()) { update
ordersToUpdate;

} if (!vehicles ToUpdate.isEmpty()) { update
vehiclesToUpdate;

} } global void finish(Database.BatchableContext bc) {

System.debug('Vehicle order batch job completed.');
```

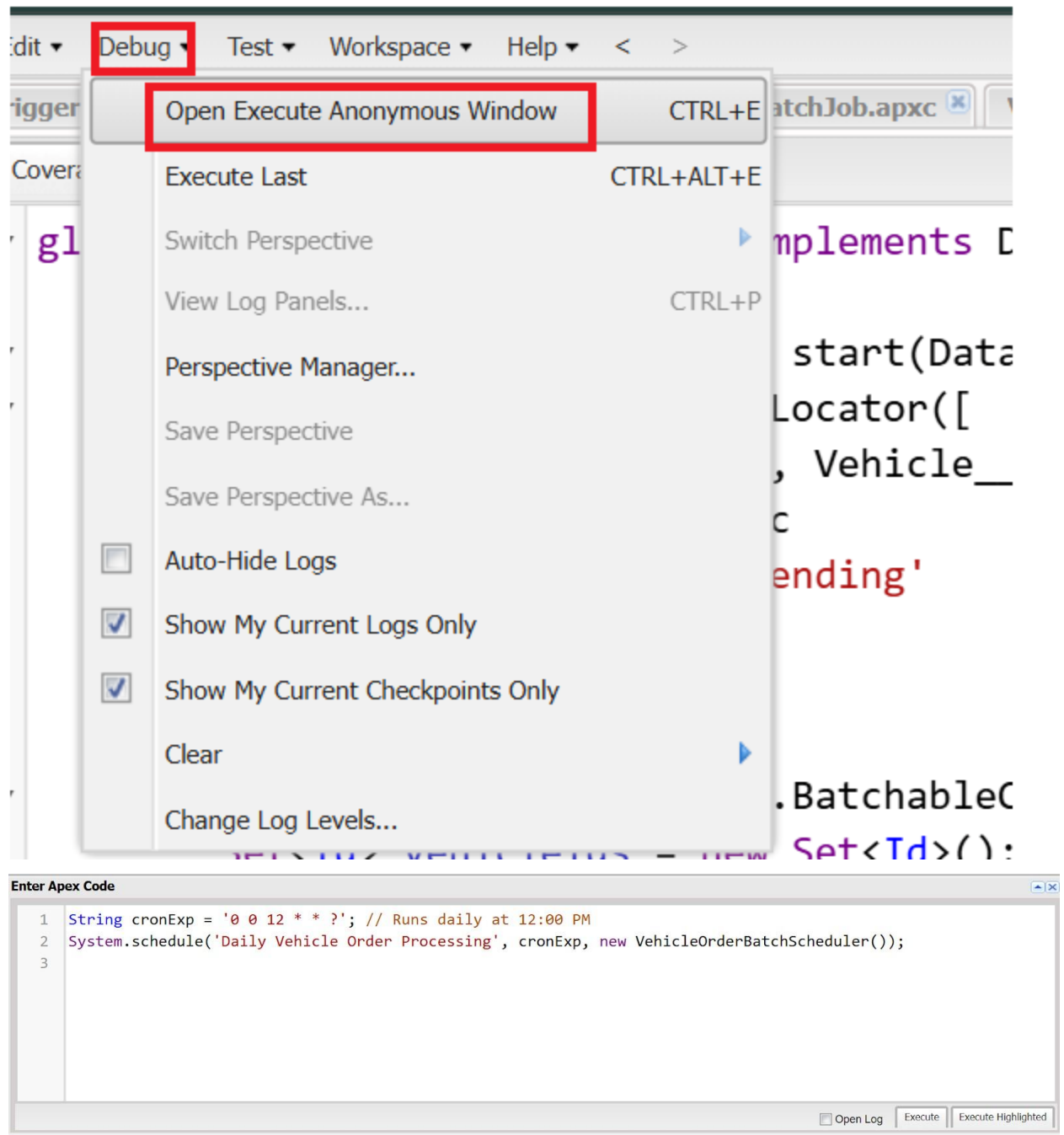
**Step 9 :** Create Schedule Class then revoked Batch Class in Schedule Class



### Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {    global void  
execute(SchedulableContext sc) {  
    VehicleOrderBatch batchJob = new VehicleOrderBatch();  
    Database.executeBatch(batchJob, 50); // 50 is the batch size  
}  
}
```

**Step 10 : Schedule the Batch Job** To run the batch job every night at midnight :



### Source Code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new
VehicleOrderBatchScheduler());
```

**Step 11 :** You Can Check where this Schedule Job is Running



7.

Screenshot of the Salesforce Setup page showing the 'Scheduled Jobs' section. The left sidebar shows the navigation menu with 'Jobs' selected. The main content area displays 'All Scheduled Jobs' with a table listing various jobs and their schedules.

| Action                   | Job Name                                             | Submitted By       | Submitted           | Started              | Next Scheduled Run   | Type                                   | Cron Trigger ID   |
|--------------------------|------------------------------------------------------|--------------------|---------------------|----------------------|----------------------|----------------------------------------|-------------------|
| Manage   Del   Pause Job | Daily Inventory Sync                                 | Trainer, Abdul     | 12/02/2025, 4:24 pm | 13/02/2025, 12:00 am | 14/02/2025, 12:00 am | Scheduled Apex                         | 00edM000000H3jb   |
| Manage   Del   Pause Job | Daily Vehicle Order Processing                       | Trainer, Abdul     | 13/02/2025, 5:14 am | 13/02/2025, 12:00 pm | 14/02/2025, 12:00 pm | Scheduled Apex                         | 00edM000000J099   |
| Del                      | Metalytics Data Loader Job for Org - 00CdM00000J0X85 | User, Integration  | 31/01/2025, 5:36 pm | 12/02/2025, 7:42 pm  | 13/02/2025, 7:42 pm  | Autonomous Data Loader Job             | 00edM000000CbnI.O |
|                          | Program Milestone Computation Cron Job               | Process, Automated | 31/01/2025, 5:36 pm | 13/02/2025, 6:30 am  | 13/02/2025, 1:29 pm  | Program Milestone Computation Cron Job | 00edM000000CbnI.M |
|                          | Program Status Update Cron Job                       | Process, Automated | 31/01/2025, 5:36 pm | 13/02/2025, 9:31 am  | 13/02/2025, 6:30 pm  | Program Status Update Cron Job         | 00edM000000CbnI.N |

# FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

## 1. Vehicle List:

Screenshot of the Salesforce 'Recently Viewed' page for 'Vehicles'. The page shows a list of 4 items: Suzuki, Honda, RevoltRV400, and Vespa. The page includes a search bar, navigation tabs, and a 'Recently Viewed' section with a list of vehicles.

WhatNext Vision Motors

Vehicles

Recently Viewed

4 items • Updated a few seconds ago

|   | Vehicle Name                         |  |
|---|--------------------------------------|--|
| 1 | <input type="checkbox"/> Suzuki      |  |
| 2 | <input type="checkbox"/> Honda       |  |
| 3 | <input type="checkbox"/> RevoltRV400 |  |
| 4 | <input type="checkbox"/> Vespa       |  |

Battery status: 55% available (plugged in)

## 2. Dealer List:

Vehicle Dealers

Recently Viewed

2 items • Updated a few seconds ago

|   | Vehicle Dealer Name      |  |
|---|--------------------------|--|
| 1 | <a href="#">Rishika</a>  |  |
| 2 | <a href="#">Priyanka</a> |  |

## 3. Vehicle Order List

Vehicle Orders

Recently Viewed

3 items • Updated a few seconds ago

|   | Vehicle_Orders Name             |  |
|---|---------------------------------|--|
| 1 | <a href="#">Pavithra</a>        |  |
| 2 | <a href="#">Priyanga sobana</a> |  |
| 3 | <a href="#">Raji</a>            |  |

## 4. Customer Record

The screenshot shows a Salesforce Lightning page for a 'Vehicle\_Customer' record named 'Raji'. The page is titled 'Developer Edition' and includes a search bar and navigation tabs for 'Vehicles', 'Vehicle Dealers', 'Vehicle Orders', 'Vehicle Customers', 'Vehicle Service Requests', 'Dashboards', and 'More'. The 'Vehicle Customers' tab is active. The record details are displayed in a table with the following information:

| Field                 | Value                            |
|-----------------------|----------------------------------|
| Vehicle_Customer Name | Raji                             |
| Owner                 | RAJALAXMI P                      |
| Customer Name         | Raji                             |
| Email                 | rajalexmpiraji@gmail.com         |
| Phone                 | (807) 262-2588                   |
| Address               | Theni                            |
| Created By            | RAJALAXMI P, 9/29/2026, 11:14 PM |
| Last Modified By      | RAJALAXMI P, 9/29/2026, 11:51 PM |

## 5. Out of Stock Validation Error Message:

The screenshot shows a Salesforce Lightning page for a 'New Vehicle Order'. The page is titled 'Vehicle Order' and includes a search bar and navigation tabs for 'Home', 'Vehicles', 'Dealers', 'Customers', 'Vehicle Orders', 'Test Drives', 'Service Requests', 'Reports', and 'Dashboards'. The 'Vehicle Orders' tab is active. The 'New Vehicle Order' form is displayed with the following information:

| Field                  | Value            |
|------------------------|------------------|
| Customer *             | Raji P           |
| Vehicle *              | Vision Sedan     |
| Quantity *             | 1                |
| Order Date *           | 15/09/2026       |
| Pending                | Pending          |
| Dealer (Auto-Assigned) | Madurai Auto Hub |

The 'Vehicle Details' section on the right shows the following information:

| Field           | Value        |
|-----------------|--------------|
| Model           | SD1          |
| Type            | Sedan        |
| Price           | ₹16,00,000   |
| Stock Available | 0            |
| Status          | Out of Stock |

A modal dialog box with a red 'X' icon and the title 'Cannot Create Order' is displayed in the center of the screen. The message reads: 'The selected vehicle is out of stock. Please choose another vehicle or wait until stock is available.' The 'OK' button is visible at the bottom right of the dialog box.

## 6. Vehicle Order Pending List:

WhatNext Vision Motors

Home

Vehicles

Dealers

Customers

Vehicle Orders

Test Drives

Service Requests

Reports

Dashboards

Q Search...

★

+

🔔

?

⚙️

🔔

👤

Vehicle Orders

Pending Orders

New

Change Owner

Export

Q Search this list...

⚙️

📄

🔄

✎

🔍

4 items • Sorted by Order Date • Filtered by Status = Pending

| <input type="checkbox"/> | <input type="checkbox"/> | Order Number ↑ | Customer Name | Vehicle      | Quantity | Order Date | Dealer (Auto-Assigned) | Status  | Expected Delivery Date | Actions              |
|--------------------------|--------------------------|----------------|---------------|--------------|----------|------------|------------------------|---------|------------------------|----------------------|
| 1                        | <input type="checkbox"/> | VO-0005        | Karthik S     | Nexa EV X1   | 1        | 14/09/2026 | Chennai Motors         | Pending | 30/09/2026             | <a href="#">View</a> |
| 2                        | <input type="checkbox"/> | VO-0003        | Divya M       | Vision Sedan | 1        | 12/09/2026 | Coimbatore Wheels      | Pending | 28/09/2026             | <a href="#">View</a> |
| 3                        | <input type="checkbox"/> | VO-0002        | Nandhini R    | Vision Prime | 1        | 11/09/2026 | Madurai Auto Hub       | Pending | 27/09/2026             | <a href="#">View</a> |
| 4                        | <input type="checkbox"/> | VO-0001        | Arun K        | Vision Sedan | 2        | 09/09/2026 | Trichy Auto Zone       | Pending | 25/09/2026             | <a href="#">View</a> |

1-4 of 4 items

⏪

⏩

1

⏪

⏩

## 7. Test Drive Scheduling & E-Mail Remainder:

WhatNext Vision Motors

Home

Vehicles

Dealers

Customers

Vehicle Orders

Test Drives

Service Requests

Reports

Dashboards

Q Search...

★

+

🔔

?

⚙️

🔔

👤

Test Drive

TD-0004

Edit

Delete

Clone

Email Reminder Sent

Test drive reminder email has been sent to raji@example.com

Details

Related

Test Drive Number

TD-0004

Status

Scheduled

Customer \*

Raji P

Assigned Dealer

Madurai Auto Hub

Vehicle \*

Vision SUV

Test Drive Type

In-Person

Preferred Date \*

20/09/2026

Notes

Interested in booking after test drive.

Preferred Time \*

10:00 AM

Location \*

Madurai Auto Hub

Cancel

Save

Test Drive Details



Vehicle Name

Vision SUV

Model

VXL

Type

SUV

Customer

Raji P

Test Drive Date

20/09/2026

Test Drive Time

10:00 AM

Location

Madurai Auto Hub

Status

Scheduled

Activity

Email

Log a Call

New Task

New Event

Test Drive Reminder

Send

Test Drive Reminder – WhatNext Vision Motors

WhatNext Vision Motors

18 Sep 2026, 09:00 AM

Hi Raji,

This is a reminder for your scheduled test drive.

Test Drive Details

Vehicle

Vision SUV

Date

20 September 2026

Time

10:00 AM

Location

Madurai Auto Hub

Type

In-Person

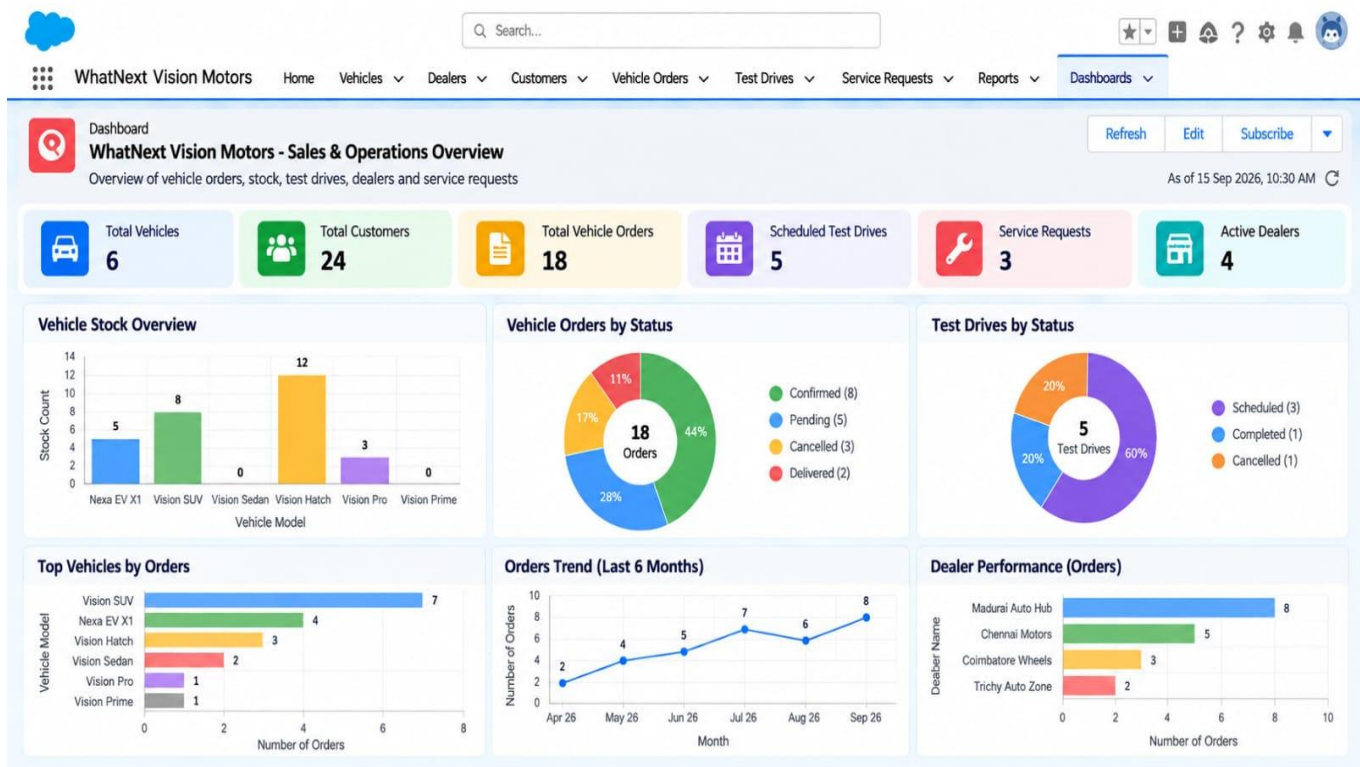
We look forward to seeing you!

For any changes, please contact us.

Regards,

WhatNext Vision Motors

## 8. Dashboard (Order & Sales Overview):



### ADVANTAGES:

- Centralized Data – Vehicle, customer, dealer, order, and service information can be managed in one Salesforce platform.
- Automated Workflows – Stock validation, dealer assignment, order processing, and reminders are automated.
- Real-Time Reporting – Reports and dashboards provide visibility into order status, dealer performance, and sales metrics.

### DISADVANTAGES:

- Salesforce Dependency – The system depends on the Salesforce platform.
- Initial Configuration Effort – Setting up objects, relationships, flows, triggers, batch jobs, and dashboards requires initial effort.

## **CONCLUSION:**

The Salesforce CRM implementation at WhatNext Vision Motors successfully modernizes vehicle ordering and dealership management. The system replaces manual and disconnected processes with a centralized and automated Salesforce platform.

## **OUTCOMES:**

- Improved vehicle order management through automated processing.
- Reduced manual intervention using workflows and Apex automation.
- Improved stock validation and order accuracy.
- Enabled automated dealer assignment based on customer location.
- Provided real-time order and sales visibility through Reports and Dashboards.
- Improved customer experience through automated test-drive reminders.
- Increased overall operational efficiency and accuracy.

Overall, the project demonstrates how Salesforce CRM can be used to streamline vehicle ordering, dealership operations, and customer engagement effectively.

## **FUTURE SCOPE:**

The future enhancements of the WhatNext Vision Motors project include:

- Mobile Ordering App – Enable customers to place and track vehicle orders through a mobile application.
- Dealer GPS Integration – Improve location-based dealer assignment using GPS.
- SMS Notifications – Send order, test-drive, and service updates through SMS.
- AI-Based Demand Forecasting – Predict vehicle demand and support better inventory planning.

## **APPENDIX:**

- Source Code: Salesforce configuration, Apex Triggers, Batch Apex, and Scheduled Apex.
- Dataset: Salesforce Objects containing vehicle, dealer, customer, order, test drive, and service request data.
- Screenshots: Salesforce application screens, workflows, automation, reports, and dashboards.
- Demo: Salesforce Developer Org implementation.
- GitHub Link: <https://github.com/PriyankaDeviPri/WhatNext-Vision-Motors-Shaping-the-Future-of-Mobility-with-Innovation-and-Excellence.git>

**Thank You**